# EE221 – Digital Logic Design

## Chapter 2 – Boolean Algebra and Logic Gates

Dr. Hussain Ali
Department of Electrical Engineering
Military College of Signals
National University of Sciences and Technology, Islamabad

Military College of Signals

Fall 2025, Dr Hussain Ali, MCS, NUST
September 08, 2025

# Table of Content


1. Basic Theorems and Properties of Boolean Algebra
   - Postulates
   - Theorems
2. Boolean Functions
3. Truth Tables
4. Logic Circuits
5. Canonical and Standard Forms
6. Other Logic Operations
7. Digital Logic Gates
8. Applications

# Basic Theorems and Properties of Boolean Algebra

## Postulates of Boolean Algebra

| | | |
|---|---|---|
| Postulate 2 | (a) $x + 0 = x$ | (b) $x \cdot 1 = x$ |
| Postulate 3 (Comm.) | (a) $x + y = y + x$ | (b) $x \cdot y = y \cdot x$ |
| Postulate 4 (Dist.) | (a) $x \cdot (y + z) = x \cdot y + x \cdot z$ | (b) $x + (y \cdot z) = (x + y) \cdot (x + z)$ |
| Postulate 5 | (a) $x + x' = 1$ | (b) $x \cdot x' = 0$ |

# Basic Theorems and Properties of Boolean Algebra

## Postulates of Boolean Algebra

| | | |
|---|---|---|
| Postulate 2 | (a) $x + 0 = x$ | (b) $x \cdot 1 = x$ |
| Postulate 3 (Comm.) | (a) $x + y = y + x$ | (b) $x \cdot y = y \cdot x$ |
| Postulate 4 (Dist.) | (a) $x \cdot (y + z) = x \cdot y + x \cdot z$ | (b) $x + (y \cdot z) = (x + y) \cdot (x + z)$ |
| Postulate 5 | (a) $x + x' = 1$ | (b) $x \cdot x' = 0$ |

Let's verify some of these postulates using truth tables

# Basic Theorems and Properties of Boolean Algebra

## Theorems of Boolean Algebra

| | | |
|---|---|---|
| Theorem 1 | (a) $x + x = x$ | (b) $x \cdot x = x$ |
| Theorem 2 | (a) $x + 1 = 1$ | (b) $x \cdot 0 = 0$ |
| Theorem 3 (involution) | $(x')' = x$ | |
| Theorem 4 (associative) | (a) $x + (y + z) = (x + y) + z$ | (b) $x \cdot (y \cdot z) = (x \cdot y)$ |
| Theorem 5 (DeMorgan) | (a) $(x + y)' = x' \cdot y'$ | (b) $(x \cdot y)' = x' + y'$ |
| Theorem 6 (absorption) | (a) $x + x \cdot y = x$ | (b) $x \cdot (x + y) = x$ |

Let's prove some of these theorems

**Operator Precedence**

1. Parentheses
2. NOT
3. AND
4. OR

- Boolean algebra deals with binary variables and logic operations
- Boolean function is an algebraic expression consisting of binary variables, the constants (0, and 1), and logic operation symbols
- A Boolean function can be equal to either 0 or 1
- Some examples of Boolean functions are given below:

**Examples:**

$$F_1 = x + y'z$$
$$F_2 = x'y'z + xyz + x'yz + xy'z$$
$$F_3 = x'yz + xz$$
$$F_4 = (x + y)'(x' + y')$$
$$F_5 = xy + x(wz + wz')$$
$$F_6 = (yz' + x'w)(xy' + zw')$$
$$F_7 = (x' + z')(x + y' + z')$$
$$F_8 = x + xz$$

Using the basic theorems and postulates of Boolean algebra, simplify the following Boolean functions:

$$
\begin{aligned}
F_2 &= x'y'z + xyz + x'yz + xy'z \\
&= x'z(y + y') + xz(y + y') \\
&= x'z + xz \\
&= (x' + x)z \\
&= z \\
F_3 &= x'yz + xz \\
&= (x'y + x)z \\
&= (x + x')(x + y)z \\
&= (x + y)z
\end{aligned}
$$

We will now simplify $F_4 = (x + y)'(x' + y')$

$$\begin{aligned} F_4 &= (x + y)'(x' + y') \\ &= x'y'(x' + y') \quad \text{DeMorgan's theorem 1} \\ &= x'y'x' + x'y'y' \\ &= x'y' + x'y' \\ &= x'y' \end{aligned}$$

**Reminder:** $(xy)' = x' + y'$ is DeMorgan's theorem 2

- A Boolean function can be represented in a truth table
- The number of rows in a truth table is $2^n$ where $n$ is the number of variables in the function/expression
- The binary combinations of the variables are obtained by counting from 0 to $2^n - 1$

# Basic Theorems and Properties of Boolean Algebra

## Truth Tables

- A Boolean function can be represented in a truth table
- The number of rows in a truth table is $2^n$ where $n$ is the number of variables in the function/expression
- The binary combinations of the variables are obtained by counting from 0 to $2^n - 1$
- Let's make the truth table for $F_1 = x + y'z$

**Example:**

| $x$ | $y$ | $z$ | $y'$ | $y'z$ | $F_1 = x + y'z$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 0 | 1 |

As another example, let's select the expression $F_4 = (x + y)'(x' + y')$ **Example**

| $x$ | $y$ | $x + y$ | $(x + y)'$ | $x'$ | $y'$ | $x' + y'$ | $F_4$ |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |

Let us now simplify the expression $F_4$

$$\begin{aligned} F_4 &= (x+y)'(x'+y') \\ &= x'y'(x'+y') \quad \text{using DeMorgan's theorem} \\ &= x'y'+x'y' = x'y' \end{aligned}$$

Let us now simplify the expression $F_4$

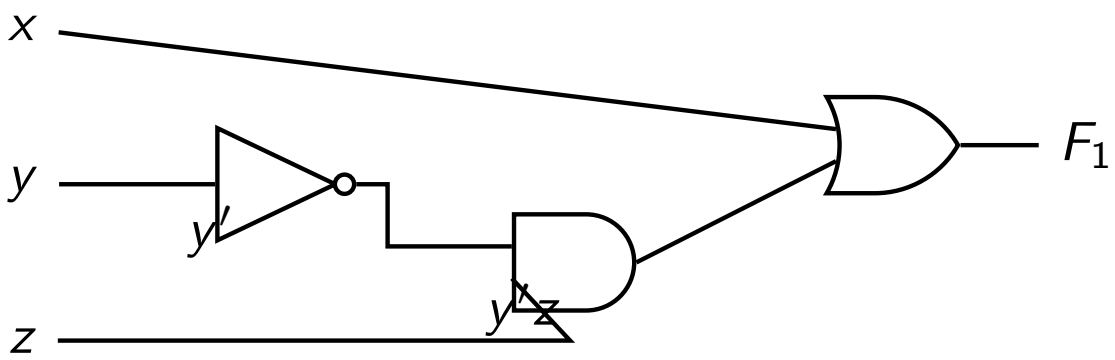
$$\begin{aligned} F_4 &= (x+y)'(x'+y') \\ &= x'y'(x'+y') \quad \text{using DeMorgan's theorem} \\ &= x'y'+x'y' = x'y' \end{aligned}$$

Let's see if the simplified version is indeed correct. To do that, we will make another truth table as shown below **Example**

| $x$ | $y$ | $x'$ | $y'$ | $x'y'$ |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 0 |

# Logic Circuits

- A Boolean function can be transformed into a circuit diagram consisting of logic gates connected in a particular structure
- The logic circuit diagram is also called a schematic
- The logic circuit diagram for $F_1 = x + y'z$ is shown below:




- There is an inverter (NOT gate) for $y$ to generate its complement $y'$
- AND gate generates $y'z$
- OR gate generates $x + y'z$

# Building Logic Circuits After Manipulating Boolean Expressions

- We have already seen that a simpler expression can be found for a given Boolean function by using the rules of Boolean algebra
- This allows one to reduce the number of gates required to implement a function
- This might also result in reduction of the number of inputs
- Motivation for you engineers: reduce the cost of the circuit
- This is an important design task
- Let's see an example

Consider a Boolean function: $F_2 = x'y'z + x'yz + xy'$
This could be simplified to: $F_2 = x'z + xy'$
The two circuits (before and after simplification are shown below)

(a) $F_2 = x'y'z + x'yz + xy'$ (b) $F_2 = xy' + x'z$

# Basic Theorems and Properties of Boolean Algebra

## Some Observations

- If we have a Boolean function, then there is only one way for it to be represented as a truth table
- A Boolean function can be simplified or even modified to another expression
- The particular expression used to represent a Boolean function dictates the structure of its equivalent logic circuit – and, therefore, the number of gates in it
- Conversely, the interconnection of logic gates will determine the logic expression
- We can use the rules of Boolean algebra to simplify expressions and thus save cost!

Let's learn a trick to simplify expressions that look impossible to simplify further

**Example**

$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z \end{aligned}$$

# Some Tricks...

Some more tricks to simplify expressions quickly
Repeated application of DeMorgan's theorem

$$(x + y + z)' = x'y'z'$$

Complement of Functions
Take the dual of the function and complement each literal

$$F_1 = x'yz' + x'y'z$$
$$F_1' = ?$$
$$\text{dual of } F_1 = (x' + y + z')(x' + y' + z)$$
$$\text{complement of each literal} = (x + y' + z)(x + y + z') = F_1'$$

- Introduction

- Introduction
- Two-variables case

- Introduction
- Two-variables case
- Three-variable case

| x | y | z | Minterms | | Maxterms | |
|---|---|---|---|---|---|---|
| | | | Term | Designation | Term | Designation |
| 0 | 0 | 0 | $x'y'z'$ | $m_0$ | $x+y+z$ | $M_0$ |
| 0 | 0 | 1 | $x'y'z$ | $m_1$ | $x+y+z'$ | $M_1$ |
| 0 | 1 | 0 | $x'yz'$ | $m_2$ | $x+y'+z$ | $M_2$ |
| 0 | 1 | 1 | $x'yz$ | $m_3$ | $x+y'+z'$ | $M_3$ |
| 1 | 0 | 0 | $xy'z'$ | $m_4$ | $x'+y+z$ | $M_4$ |
| 1 | 0 | 1 | $xy'z$ | $m_5$ | $x'+y+z'$ | $M_5$ |
| 1 | 1 | 0 | $xyz'$ | $m_6$ | $x'+y'+z$ | $M_6$ |
| 1 | 1 | 1 | $xyz$ | $m_7$ | $x'+y'+z'$ | $M_7$ |

- Minterms are also called standard products
- Maxterms are also called standard sums

- Sum of Minterms (Sum = OR)
- Product of Maxterms (Product = AND)
- Conversion between Canonical Forms

| x | y | z | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

$$F = xy + x'z$$

$$F(x, y, z) = x'y'z + x'yz + xyz' + xyz = \Sigma(1, 3, 6, 7)$$

$$F(x, y, z) = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') = \Pi(0, 2, 4, 5)$$

# Standard Forms

- In Canonical forms, functions have terms that contain all variables
- In Standard forms, terms of a function may contain one, two, or any number of literals/variables
- Sum of Products (SOP)
- Product of Sums (POS)

**Examples:**

SOP:

$$F_1 = x'yz + x'y' + xy + z$$
$$F_2 = x + y' + xyz'$$
$$F_3 = xyz$$

POS:

$$F_4 = (x' + y + z)(x' + y')(x + y + z)$$
$$F_5 = (x + y')(x + y + z')z$$

Non-standard:

$$F_6 = x'(y + z) + z(x'y' + x) + z$$

# Standard Forms

- Standard forms result in two-level implementation unlike non-standard form
- However, there could be a downside $\rightarrow$ number of inputs to a gate could be impractical
- We prefer two-level implementation whenever practical $\rightarrow$ least amount of delay

  (a) $AB + CD + E$ (b) $AB + CD + CE$

- To draw a two-level diagram, function must be written in a standard form (SOP or POS)

# Basic Theorems and Properties of Boolean Algebra

## Other Logic Operations

| x | y | F0 | F1 | F2 | F3 | F4 | F5 | F6 | F7 | F8 | F9 | F10 | F11 | F12 | F13 | F14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |

- Two functions that produce a constant 0 or 1.
- Four functions with unary operations: complement and transfer
- Ten functions with binary operators that define eight different operations: AND, OR, NAND, NOR, exclusive-OR, equivalence, inhibition, and implication

# Basic Theorems and Properties of Boolean Algebra

## Other Logic Operations

| Boolean Functions | Operator Symbol | Name | Comments |
|---|---|---|---|
| $F_0 = 0$ | | Null | Binary constant 0 |
| $F_1 = xy$ | $x \cdot y$ | AND | $x$ and $y$ |
| $F_2 = xy'$ | $x/y$ | Inhibition | $x$, but not $y$ |
| $F_3 = x$ | | Transfer | $x$ |
| $F_4 = x'y$ | $y/x$ | Inhibition | $y$, but not $x$ |
| $F_5 = y$ | | Transfer | $y$ |
| $F_6 = xy' + x'y$ | $x \oplus y$ | Exclusive-OR | $x$ or $y$, but not both |
| $F_7 = x + y$ | $x + y$ | OR | |
| $F_8 = (x + y)'$ | $x \downarrow y$ | NOR | Not-OR |
| $F_9 = xy + x'y'$ | $(x \oplus y)'$ | Equivalence | $x$ equals $y$ |
| $F_{10} = y'$ | $y'$ | Complement | Not $y$ |
| $F_{11} = x + y'$ | $x \subset y$ | Implication | If $y$, then $x$ |
| $F_{12} = x'$ | $x'$ | Complement | Not $x$ |
| $F_{13} = x' + y$ | $x \supset y$ | Implication | If $x$, then $y$ |
| $F_{14} = (xy)'$ | $x \uparrow y$ | NAND | Not-AND |
| $F_{15} = 1$ | | Identity | Binary constant 1 |

- NOR function is the complement of OR function (not-OR)
- NAND function is the complement of AND function (not-AND)
- XOR (exclusive OR) is similar to OR but excludes the combination of both x and y being equal to 1
- Equivalence is 1 when x and y are equal
- Equivalence is complement of XOR, therefore also called exclusive NOR (XNOR)

# Basic Theorems and Properties of Boolean Algebra

## Digital Logic Gates

- A manufacturing plant uses two tanks to store a liquid chemical that is used in a manufacturing process.
- Each tank has a sensor that detects when the chemical level drops to 25% of full.
- The sensors produce a 5 V signal (logic 1) when the tanks are more than 25% full and 0 V (logic 0) if less.
- Green light switches ON when both tanks are above the 25% level.
- Use logic gates to implement this function?
- Let's say we need the output of our logic circuit to be 1 if both tanks are above the 25% level.

# Basic Theorems and Properties of Boolean Algebra

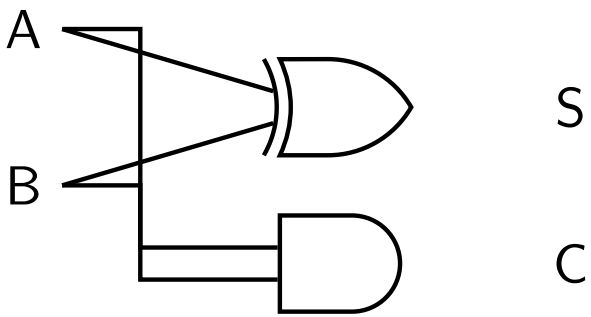
## Application I

- Consider the manufacturing plant of Application I.
- Now we need to display a red light when the chemical level in at least one of the tanks drops to 25% of full.
- The sensors produce a 5 V signal (logic 1) when the tanks are more than 25% full and 0 V (logic 0) if less.
- Use logic gates to implement this function?
- Let's say we need the output of our logic circuit to be 1 if at least one of the tanks drops to 25% of full.

# Basic Theorems and Properties of Boolean Algebra

## Application II

To add two bits, we can use a combination of a XOR and an AND gate as follows




| A | B | Carry | Sum |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

# End of Chapter 2